DATE	
PROGRAM-01	Implement Operations On Singly Linked List- Create,Insert,Delete,Traverse

```
#include <malloc.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct sllist {
   int data;
   struct sllist *next;
} node;

node *head = NULL, *temp = NULL, *cur;

void create();
void insert();
void insert_beg();
void insert_end();
void insert_middle();
void todelete();
void display();

int main() {
   int choice = 0;
   while (1) {
      printf("\nSingly linked list operations");
      printf("\n----------------------------");
      printf("\n1. Create");
      printf("\n2. Insertion");
      printf("\n3. Traversal");
      printf("\n4. Deletion");
      printf("\n5. Exit");
      printf("\nEnter your choice: ");
      scanf("%d", &choice);


switch (choice) {
         case 1:
            create();
            break;
         case 2:
            insert();
            break;
         case 3:
            display();
```

```
            break;
        case 4:
            todelete();
            break;
        case 5:
            printf("\nEnd of the list operations");
            exit(0);
        default:
            printf("\nWrong choice---> Try again");
            break;
    }
  }
}

void create() {
  int n, i;
  node *cur;
  printf("\nHow many nodes to create: ");
  scanf("%d", &n);

  for (i = 1; i <= n; i++) {
    temp = (node*)malloc(sizeof(node));
    printf("\nEnter data: ");
    scanf("%d", &temp->data);
    temp->next = NULL;

    if (head == NULL) {
      head = temp;
    } else {
      cur = head;
      while (cur->next != NULL) {

cur = cur->next;
      }
      cur->next = temp;
    }
  }
}

void display() {
  if (head == NULL) {
    printf("\nEmpty List");
  } else {
    cur = head;
    printf("\nLinked list elements");
    printf("\n-----------------------------");
```

```
        while (cur != NULL) {
            printf("\n%d", cur->data);
            cur = cur->next;
        }
        printf("\n---------------------");
    }
}

void todelete() {
    int ele;
    node *prev = head;

    if (head == NULL) {
        printf("\nEmpty list --> Deletion is not possible");
        return;
    }

    display();
    printf("\nWhat to delete? ");
    scanf("%d", &ele);

    cur = head;
    while (cur->data != ele && cur->next != NULL) {
        prev = cur;
        cur = cur->next;
    }

    if (cur->data == ele) {
        if (cur == head) {
            head = cur->next;
        } else {
            prev->next = cur->next;
            free(cur);
        }
        if (cur->next == NULL) {
            printf("\nElement to be deleted is not in the list");
        }
    }
}

void insert() {
    int option = 0;
    temp = malloc(sizeof(node));
    printf("\nEnter data: ");
    scanf("%d", &temp->data);
    temp->next = NULL;
```

```
    printf("\nInsertion menu:");
    printf("\n------------------");
    printf("\n1. Beginning");
    printf("\n2. Middle");
    printf("\n3. End");
    printf("\nWhere to insert??: ");
    scanf("%d", &option);

    switch(option) {
        case 1:
            insert_beg();
            break;
        case 2:
            if (head == NULL) {
                printf("\nInsertion in middle is not possible");
                return;
            }
            display();
            insert_middle();
            break;
        case 3:
            insert_end();
            break;
        default:
            printf("\nPlease enter the correct position");
    }
}

void insert_beg() {
    temp->next = head;
    head = temp;
}

void insert_end() {
    cur = head;
    while (cur->next != NULL) {
        cur = cur->next;
    }
    cur->next = temp;
}

void insert_middle() {
    int ele;
    printf("\nAfter which node??? ");
    scanf("%d", &ele);
```

```
    cur = head;
    while (cur->data != ele && cur->next != NULL) {
        cur = cur->next;
    }

    if (cur->data == ele) {
        temp->next = cur->next;
        cur->next = temp;
    }

    if (cur->next == NULL) {
        printf("\n%d is not in the list", ele);
    }
}
```

OUTPUT:

EMPTY LIST:-

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:3

empty list
```

CREATION:

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:1

how many nodes to create4

enter data:4

enter data:5

enter data:6
```

```
enter data:7

Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:3
linked list elements

4
5
6
7
```

INSERTION:- (INSERT IN BEGINNING)

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:2

enter data:5
insertion menu:

1.beginning
2.middle
3.end
where to insert??:1
```

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:3
linked list elements

5
4
5
6
7
```

(INSERTION IN MIDDLE):

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:2

enter data:9
insertion menu:

1.beginning
2.middle
3.end
where to insert??:2
```

```
linked list elements

5
4
5
6
7

after which node??? 5

Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
```

```
enter your choice:3
linked list elements

5
9
4
5
6
7
```

(INSERTION IN THE END):

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:2

enter data:10
insertion menu:

1.beginning
2.middle
3.end
where to insert??:3
```

```
Singly linked list operations

1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:3
linked list elements

5
9
4
5
6
7
10
```

WRONG CHOICE:-

```
Singly linked list operations
_____________________________
1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:7

wrong choice--> try again
```

DELETION:-

```
Singly linked list operations
_____________________________
1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:4

empty list --> deletion is not possible
```

```
Singly linked list operations
_____________________________
1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:4
linked list elements
____________________
5
9
4
5
6
7
10
```

```
_____________________________
what to delete??15

element not found in the list
Singly linked list operations
_____________________________
1.Creations:
2.Insertion:
3.Traversal:
4.deletion:
5.Exit
enter your choice:4
```

```
linked list elements
_____________
 5
 9
 4
 5
 6
 7
 10
_____________________
 what to delete??4
```

```
 Singly linked list operations
_____________________________________
 1.Creations:
 2.Insertion:
 3.Traversal:
 4.deletion:
 5.Exit
 enter your choice:3
linked list elements
_____________________
 5
 9
 5
 6
 7
 10
```

EXIT:-

```
 Singly linked list operations
_____________________________
 1.Creations:
 2.Insertion:
 3.Traversal:
 4.deletion:
 5.Exit
 enter your choice:5

 end of the list oprations
```

Date:	
Program 2	Program to implement operations on Doubly Linked List: insert, delete, traverse

```
#include <malloc.h>
typedef struct dllist
{
   int data;
   struct dllist *prev;
   struct dllist *next;
} node;

node *head = NULL, *temp = NULL, *cur = NULL;

void create();
void insert();
void insert_beg();
void insert_end();
void insert_middle();
void todelete();
void display();

int main()
{
   int choice = 0;

   while (1)
   {
      printf("\n Doubly Linked List Operations");
      printf("\n_____");
      printf("\n 1. Creation");
      printf("\n 2. Insertion");
      printf("\n 3. Traversal");
      printf("\n 4. Deletion");
      printf("\n 5. Exit");
      printf("\n Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1: create(); break;
         case 2: insert(); break;
```

```
            case 3: display(); break;
            case 4: todelete(); break;
            case 5:
                printf("\n End of list operations");
                exit(0);
            default:
                printf("\n Wrong choice --> Try again");
        }
    }
}

void create()
{
    int n, i;
    printf("\n How many nodes to create: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        temp = (node *)malloc(sizeof(node));
        printf(" Enter data: ");
        scanf("%d", &temp->data);

        temp->prev = NULL;
        temp->next = NULL;

        if (head == NULL)
        {
            head = temp;
        }
        else
        {
            cur = head;
            while (cur->next != NULL)
                cur = cur->next;

            cur->next = temp;
            temp->prev = cur;
        }
    }
}

void display()
{
    if (head == NULL)
    {
```

```
        printf("\n Empty list");
        return;
    }

    cur = head;
    printf("\n Doubly Linked List:");
    printf("\n_______________________");
    while (cur != NULL)
    {
        printf("\n %d", cur->data);
        cur = cur->next;
    }
    printf("\n_______________________");
}

void todelete()
{
    int ele;

    if (head == NULL)
    {
        printf("\n Empty list --> Deletion not possible");
        return;
    }

    display();
    printf("\n Enter element to delete: ");
    scanf("%d", &ele);

    cur = head;
    while (cur != NULL && cur->data != ele)
        cur = cur->next;

    if (cur == NULL)
    {
        printf("\n Element not found");
        return;
    }

    if (cur == head)
        head = cur->next;

    if (cur->next != NULL)
        cur->next->prev = cur->prev;

    if (cur->prev != NULL)
```

```
        cur->prev->next = cur->next;

    free(cur);
    printf("\n Node deleted successfully");
}

void insert()
{
    int option = 0;

    temp = (node *)malloc(sizeof(node));
    printf("\n Enter data: ");
    scanf("%d", &temp->data);

    temp->prev = NULL;
    temp->next = NULL;

    printf("\n Insertion Menu");
    printf("\n___________________________");
    printf("\n 1. Beginning");
    printf("\n 2. Middle");
    printf("\n 3. End");
    printf("\n Where to insert: ");
    scanf("%d", &option);

    switch (option)
    {
        case 1: insert_beg(); break;
        case 2:
            if (head == NULL)
            {
                printf("\n Middle insertion not possible");
                return;
            }
            display();
            insert_middle();
            break;
        case 3: insert_end(); break;
        default:
            printf("\n Please enter correct option");
    }
}

void insert_beg()
{
    if (head == NULL)
```

```
    {
        head = temp;
    }
    else
    {
        temp->next = head;
        head->prev = temp;
        head = temp;
    }
}

void insert_end()
{
    if (head == NULL)
    {
        head = temp;
        return;
    }
     cur = head;
    while (cur->next != NULL)
        cur = cur->next;

    cur->next = temp;
    temp->prev = cur;
}

void insert_middle()
{
    int ele;
    printf("\n Insert after which element: ");
    scanf("%d", &ele);
     cur = head;
    while (cur != NULL && cur->data != ele)
        cur = cur->next;

    if (cur == NULL)
    { printf("\n %d not found in the list", ele);
        return;
    }
    temp->next = cur->next;
    temp->prev = cur;
     if (cur->next != NULL)
        cur->next->prev = temp;

    cur->next = temp;
}
```

OUTPUT:

EMPTY LIST:

```
 Doubly Linked List Operations
_____________________________
 1. Creation
 2. Insertion
 3. Traversal
 4. Deletion
 5. Exit
 Enter your choice: 3

 Empty list
```

CREATION:

```
 Doubly Linked List Operations
_____________________________
 1. Creation
 2. Insertion
 3. Traversal
 4. Deletion
 5. Exit
 Enter your choice: 1

 How many nodes to create: 4
 Enter data: 7
 Enter data:
5
 Enter data: 6
 Enter data: 2
```

INSERTION IN THE BEGINING:

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 2

Enter data: 1

Insertion Menu
_____________________
1. Beginning
2. Middle
3. End
Where to insert: 1
```

INSERTION IN THE MIDDLE:

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 2

Enter data: 6

Insertion Menu
_____________________
1. Beginning
2. Middle
3. End
Where to insert: 2
```

```
Doubly Linked List:
___________________
1
7
5
6
2
___________________
Insert after which element: 7
```

INSERTION IN THE END

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 2

Enter data: 9

Insertion Menu
_____________________
1. Beginning
2. Middle
3. End
Where to insert: 3
```

```
Doubly Linked List:
___________________
1
7
6
5
6
2
9
___________________
```

DISPLAY:

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 3
```

```
Doubly Linked List:
___________________
1
7
6
5
6
2
9
___________________
Enter element to delete: 11

Element not found
```

WRONG CHOICE:

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 7

Wrong choice --> Try again
```

DELETION:

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 4
```

```
Doubly Linked List:
___________________
1
7
6
5
6
2
9
___________________
Enter element to delete: 11

Element not found
```

```
Doubly Linked List Operations
_____________________________
1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 4
```

```
Doubly Linked List:
___________________
1
7
6
5
6
2
9
___________________
Enter element to delete: 6

Node deleted successfully
```

EXIT:

```
Doubly Linked List Operations

1. Creation
2. Insertion
3. Traversal
4. Deletion
5. Exit
Enter your choice: 5

End of list operations
```

Date:	
Program 3	Program to implement operations on Circular Linked List: insert, delete, traverse

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node *head = NULL;

// -------- CREATE MULTIPLE NODES ----------
void create() {
   int n, value;

   if (head != NULL) {
      printf("List already created!\n");
      return;
   }

   printf("How many nodes do you want to enter? ");
   scanf("%d", &n);

   for (int i = 1; i <= n; i++) {
      printf("Enter data for node %d: ", i);
      scanf("%d", &value);

      struct Node *newNode = malloc(sizeof(struct Node));
      newNode->data = value;

      if (head == NULL) {
         head = newNode;
         newNode->next = head;
      }
      else {
         struct Node *temp = head;
         while (temp->next != head)
            temp = temp->next;

         temp->next = newNode;
```

```
            newNode->next = head;
        }
    }

    printf("List created with %d nodes.\n", n);
}

// -------- INSERT ----------
void insert() {
    if (head == NULL) {
        printf("List empty. First create.\n");
        return;
    }

    int choice, value, pos;
    printf("\nInsert Menu:\n1. Beginning\n2. End\n3. Middle (after position)\nEnter choice: ");
    scanf("%d", &choice);

    printf("Enter data: ");
    scanf("%d", &value);

    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = value;

    // Insert at beginning
    if (choice == 1) {
        struct Node *temp = head;
        while (temp->next != head)
            temp = temp->next;

        temp->next = newNode;
        newNode->next = head;
        head = newNode;

        printf("Inserted at beginning.\n");
    }

    // Insert at end
    else if (choice == 2) {
        struct Node *temp = head;
        while (temp->next != head)
            temp = temp->next;

        temp->next = newNode;
        newNode->next = head;
```

```
        printf("Inserted at end.\n");
    }

    // Insert middle
    else if (choice == 3) {
        printf("Insert after which position? ");
        scanf("%d", &pos);

        struct Node *temp = head;
        for (int i = 1; i < pos; i++) {
            temp = temp->next;
            if (temp == head) {
                printf("Position out of range.\n");
                return;
            }
        }

        newNode->next = temp->next;
        temp->next = newNode;

        printf("Inserted after position %d.\n", pos);
    }

    else {
        printf("Invalid choice.\n");
    }
}

// -------- TRAVERSAL ----------
void traversal() {
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }

    struct Node *temp = head;
    printf("Circular Linked List: ");

    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);

    printf("\n");
}
```

```
// -------- DELETE BY VALUE ONLY ----------
void deletion() {
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }

    int value;
    printf("Enter value to delete: ");
    scanf("%d", &value);

    struct Node *temp = head, *prev = NULL;

    // If deleting head node
    if (head->data == value) {
        struct Node *last = head;

        while (last->next != head)
            last = last->next;

        // only 1 node
        if (head->next == head) {
            free(head);
            head = NULL;
        } else {
            last->next = head->next;
            temp = head;
            head = head->next;
            free(temp);
        }

        printf("Value %d deleted.\n", value);
        return;
    }

    // Search in remaining nodes
    do {
        prev = temp;
        temp = temp->next;

        if (temp->data == value) {
            prev->next = temp->next;
            free(temp);
            printf("Value %d deleted.\n", value);
            return;
        }
```

```
    } while (temp != head);

    printf("Value not found.\n");
}

// -------- MAIN ----------
int main() {
    int choice;

    while (1) {
        printf("\n--- Circular Linked List Menu ---\n");
        printf("1. Create\n2. Insert\n3. Traversal\n4. Delete \n5. Exit\n");
        printf("---------------------------------\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create(); break;
            case 2: insert(); break;
            case 3: traversal(); break;
            case 4: deletion(); break;
            case 5: exit(0);
            default: printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

OUTPUT:

EMPTY LIST

```
--- Circular Linked List  ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 3
List empty.

--- Circular Linked List  ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 4
List empty.
```

CREATION:

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 1
How many nodes do you want to enter? 4
Enter data for node 1: 34
Enter data for node 2: 45
Enter data for node 3: 56
Enter data for node 4: 67
List created with 4 nodes.
```

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 3
Circular Linked List: 34 45 56 67
```

INSERTION(BEG):

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
----------------------------------
Enter your choice: 2

Insert Menu:
1. Beginning
2. End
3. Middle (after position)
Enter choice: 1
Enter data: 45
Inserted at beginning.
```

INSERTION(END):

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
----------------------------------
Enter your choice: 2

Insert Menu:
1. Beginning
2. End
3. Middle (after position)
Enter choice: 2
Enter data: 56
Inserted at end.
```

INSERTION(MIDDLE):

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 2

Insert Menu:
1. Beginning
2. End
3. Middle (after position)
Enter choice: 3
Enter data: 89
Insert after which position? 3
Inserted after position 3.
```

TRAVERSE:

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 3
Circular Linked List: 45 34 45 89 56 67 56
```

DELETION:

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 4
Enter value to delete: 67
Value 67 deleted.

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 3
Circular Linked List: 45 34 45 89 56 56
```

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 4
Enter value to delete: 90
Value not found.
```

EXIT:

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Traversal
4. Delete
5. Exit
---------------------------------
Enter your choice: 5


=== Code Execution Successful ===
```

DATE	
Program 4	Program to implement stack operations using pointers (push, pop, process)

```
#include <malloc.h>
typedef struct stack
{
   int data;
   struct stack*next;
} node;
node*top=NULL,*temp;

void push(int);
int pop();
void display();
int main()
{
   int choice=0,item,i,n,ele[100];
   while(1)
   {
      printf("\n stack operstions");
      printf("\n------------------");
      printf("\n 1.push");
      printf("\n 2.pop");
      printf("\n 3.display");
      printf("\n4.exit");
      printf("\n enter your choice");
      scanf("%d",&choice);
      switch (choice)
      {
         case 1:printf("\n enter the number of items to push");
               scanf("%d",&n);
               printf("\n enter the elements");
               for (i=0;i<n;i++)
               {
                scanf("%d",&ele[i]);
                push(ele[i]);
                }
                break;

         case 2:if(top==NULL)
               printf("\n empty stack");
               else
```

```
            printf("\n popped item=%d",pop());
            break;
        case 3:display();
            break;
        case 4:printf("\n ending stack operation");
            getch();
            exit(0);
       default:printf("\n invalid choice");
       return 0;
      }
   }
}
void push(int item)
{
   temp=(node *)malloc(sizeof(node));
   temp->data=item;
   temp->next=NULL;
   temp->next=top;
   top=temp;
}
int pop()
{
   int data;
   temp=top;
   top=temp->next;
   data=temp->data;
   free(temp);
   return data;
}
void display()
{
   if(top==NULL)
   printf("\n empty stack");
   else
   {
      printf("\n stack members");
      printf("\n-----------------");
      temp=top;
      while (temp!=NULL)
      {
         printf("\n %d",temp->data);
         temp=temp->next;
      }
   }
}
```

OUTPUT:

EMPTY STACK:

```
 stack operations
------------------
 1.push
 2.pop
 3.display
4.exit
 enter your choice2

 empty stack
 stack operations
------------------
 1.push
 2.pop
 3.display
4.exit
 enter your choice3

 empty stack
```

PUSH:

```
 stack operations
------------------
 1.push
 2.pop
 3.display
4.exit
 enter your choice1

 enter the number of items to push4

 enter the elements12
23
34
45
```

POP:

```
 stack operations
-----------------
 1.push
 2.pop
 3.display
4.exit
 enter your choice2

 popped item=45
```

DISPLAY:

```
 stack operations
-----------------
 1.push
 2.pop
 3.display
4.exit
 enter your choice3

 stack members
----------------
 34
 23
 12
```

EXIT:

```
 stack operations
-----------------
 1.push
 2.pop
 3.display
4.exit
 enter your choice4

 ending stack operation
```

Date:	
Program 5	Program to implement operations on queue using pointers(insertion,deletion,process the queue)

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Enqueue operation
void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow!\n");
    } else {
        if (front == -1)
            front = 0;
        queue[++rear] = value;
        printf("%d inserted into the queue.\n", value);
    }
}

// Dequeue operation
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
    } else {
        printf("%d removed from the queue.\n", queue[front++]);
    }
}

// Display queue
void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
    } else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
```

```
    }
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Queue Operations ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4: printf("\n exit ");
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }
}
```

OUTPUT:

QUEUE UNDERFLOW

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Queue Underflow!

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue is empty.
```

ENQUEUE:

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 45
45 inserted into the queue.

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 56
56 inserted into the queue.
```

DEQUEUE:

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
45 removed from the queue.
```

DISPLAY:

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 56
```

EXIT:

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4

 exit
```

Date:	
Program 6	Program to implement operations on double ended queue using pointers(a) insert in the beginning b) insert in the end c)delete from beginning d)delete from end.

```
#include <stdio.h>
#define MAX 5

int deque[MAX];
int front = -1, rear = -1;

int isFull() {
   return (front == 0 && rear == MAX - 1) || (front == rear + 1);
}

int isEmpty() {
   return front == -1;
}

void insertFront(int item) {
   if (isFull()) {
      printf("Deque Overflow\n");
      return;
   }

   if (isEmpty())
      front = rear = 0;
   else if (front == 0)
      front = MAX - 1;
   else
      front--;

   deque[front] = item;
   printf("insertion in the front. Successfull!!! \n");

}

void insertRear(int item) {
   if (isFull()) {
      printf("Deque Overflow\n");
      return;
   }

   if (isEmpty())
      front = rear = 0;
```

```
    else if (rear == MAX - 1)
        rear = 0;
    else
        rear++;

    deque[rear] = item;
    printf("insertion in the rear. successfull!!!");
}
void deleteFront() {
    if (isEmpty()) {
        printf("Deque Underflow\n");
        return;
    }

    printf("Deleted element: %d\n", deque[front]);

    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}
void deleteRear() {
    if (isEmpty()) {
        printf("Deque Underflow\n");
        return;
    }
printf("Deleted element: %d\n", deque[rear]);

    if (front == rear)
        front = rear = -1;
    else if (rear == 0)
        rear = MAX - 1;
    else
        rear--;
}
void display() {
    int i;

    if (isEmpty()) {
        printf("Deque is empty\n");
        return;
    }

    printf("Deque elements: ");
    i = front;
    while (1) {
```

```
        printf("%d ", deque[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    int choice, item;

    do {
            printf("---Double Ended Queue-------");
        printf("\n1.Insert Front \n2.Insert Rear");
        printf("\n3.Delete Front  \n4.Delete Rear");
        printf("\n5.Display      \n 6.Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: printf("enter data:");
                scanf("%d", &item);
                insertFront(item);
                break;
            case 2:printf("enter data:");
                scanf("%d", &item);
                insertRear(item);
                break;
            case 3:
                deleteFront();
                break;
            case 4:
                deleteRear();
                break;
            case 5:
                display();
                break;
            case 6:
                break;
            default:
                printf("Invalid choice\n");
        }
    } while (choice != 6);

    return 0;
}
```

OUTPUT:

DEQUE UNDERFLOW:

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 3
Deque Underflow
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 4
Deque Underflow
```

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 5
Deque is empty
```

INSERTION IN THE FRONT:

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 1
enter data:56
insertion in the front. Successfull!!!
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 1
enter data:56
insertion in the front. Successfull!!!
```

INSERTION IN THE REAR:

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 2
enter data:34
insertion in the rear. successfull!!!---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 2
enter data:89
insertion in the rear. successfull!!!---Double Ended Queue-------
```

DELETION IN THE FRONT:

```
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 3
Deleted element: 56
```

DELETION IN THE REAR:

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 4
Deleted element: 89
```

DISPLAY:

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 5
Deque elements: 56 34
```

EXIT:

```
---Double Ended Queue-------
1.Insert Front
2.Insert Rear
3.Delete Front
4.Delete Rear
5.Display
 6.Exit
Enter choice: 6


=== Code Execution Successful ===
```

Date:	
Program 7	Program to implement conversion of infix expression to postfix form.

```
#include <malloc.h>
typedef struct opstack
{
	char data;
	struct opstack*next;
}node;
node*TOP=NULL,*temp=NULL;
void push(char);
char pop();
int stackpriority(char);
int infixpriority(char);
void main()
{
	char infix[50],postfix[50],ch,ch1;
	int i=0,j=0;
	printf("\n conversion of infix expression to postfix expression");
	printf("\n-----------------------------------");
	printf("\n enter on infix expression with only +,-,*,/,(,) and single operand:");
	gets(infix);
	strcpy(postfix,"");
	while (infix[i]!='\0')
	{
		ch=infix[i];
		if(('A'<=ch&&ch<='Z')||('a'<=ch&&ch<='z'))
		postfix[j++]=ch;
		else if(ch=='(')
		{
			push(ch);
		}
		else if(ch==')')
		{
			ch1=pop();
			while(ch1!='('&&ch1!='\0')
			{
				postfix[j++]=ch1;
				ch1=pop();
			}
		}
		else
```

```
                {
                while(TOP!=NULL && stackpriority(TOP->data)>=infixpriority(ch))
                        {
                                postfix[j++]=pop();
                        }
                        push(ch);
                }
                i++;
        }
        while(TOP!=NULL)
        {
                ch1=pop();
                postfix[j++]=ch1;
        }
        postfix[j]='\0';
        printf("\n the postfix expression is %s",postfix);
}
void push(char ch)
{
        node*temp;
        temp=(node *)malloc(sizeof(node));
        temp->data=ch;
        temp->next=TOP;
        TOP=temp;
}
char pop()
{
        node*temp;
        char x;
        if(TOP==NULL)
        return '\0';
        temp=TOP;
        x=temp->data;
        TOP=TOP->next;
        free(temp);
        return x;
}
int infixpriority(char ch)
{
        if(ch == '+' || ch == '-')
        return 1;
        else if(ch == '*' || ch == '/')
        return 2;
        else if(ch == '(')
        return 3;
        else
```

```
        return 0;
}
int stackpriority(char ch)
{
        if(ch == '+' || ch == '-')
        return 1;
        else if(ch == '*' || ch == '/')
        return 2;
        else if(ch == '(')
        return 0;
        else
        return 100;
}
```

OUTPUT:

```
Enter infix expression: a+b-c*d/f
Postfix expression: ab+cd*f/-


=== Code Execution Successful ===
```

Date:	
Program 8	Program to implement evaluation of postfix expression.

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

/* Push */
void push(int value) {
   stack[++top] = value;
}

/* Pop */
int pop() {
   return stack[top--];
}

int main() {
   char postfix[MAX], ch;
   int i, op1, op2, value;

   printf("Enter postfix expression: ");
   scanf("%s", postfix);

   for (i = 0; postfix[i] != '\0'; i++) {
      ch = postfix[i];

      /* If operand (variable) */
      if (isalpha(ch)) {
         printf("Enter value of %c: ", ch);
         scanf("%d", &value);
         push(value);
      }
      /* If operator */
      else {
         op2 = pop();
         op1 = pop();

         switch (ch) {
```

```
            case '+':
                push(op1 + op2);
                break;
            case '-':
                push(op1 - op2);
                break;
            case '*':
                push(op1 * op2);
                break;
            case '/':
                push(op1 / op2);
                break;
            default:
                printf("Invalid operator\n");
                return 1;
        }
    }
}

printf("Result = %d\n", pop());
return 0;
}
```

OUTPUT:

```
Enter postfix expression: abc+-cd*
Enter value of a: 2
Enter value of b: 3
Enter value of c:
4
Enter value of c: 6
Enter value of d: 7
Result = 42
```

Date:	
Program 9	Program to implement sort algorithms (Any two – on rotation) (Insertion sort, Merge sort, Quick sort, Heap sort, Radix sort)

```
#include <stdio.h>

// Function to print array
void printArray(int arr[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// ---------------- INSERTION SORT ----------------
void insertionSort(int arr[], int n) {
    for(int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// ---------------- MERGE SORT ----------------
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for(i = 0; i < n1; i++) L[i] = arr[l + i];
    for(j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;

    while(i < n1 && j < n2) {
        if(L[i] <= R[j])
            arr[k++] = L[i++];
        else
```

```
            arr[k++] = R[j++];
    }

    while(i < n1) arr[k++] = L[i++];
    while(j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

// ---------------- HEAP SORT ----------------
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && arr[left] > arr[largest])
        largest = left;

    if(right < n && arr[right] > arr[largest])
        largest = right;

    if(largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for(int i = n-1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
```

```
        heapify(arr, i, 0);
    }
}

// ---------------- QUICK SORT ----------------
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// ---------------- MAIN MENU ----------------
int main() {
    int arr[100], n, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\n--- MENU ---\n");
```

```
        printf("1. Insertion Sort\n");
        printf("2. Merge Sort\n");
        printf("3. Heap Sort\n");
        printf("4. Quick Sort\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                insertionSort(arr, n);
                printf("Sorted using Insertion Sort:\n");
                printArray(arr, n);
                break;

            case 2:
                mergeSort(arr, 0, n - 1);
                printf("Sorted using Merge Sort:\n");
                printArray(arr, n);
                break;

            case 3:
                heapSort(arr, n);
                printf("Sorted using Heap Sort:\n");
                printArray(arr, n);
                break;

            case 4:
                quickSort(arr, 0, n - 1);
                printf("Sorted using Quick Sort:\n");
                printArray(arr, n);
                break;

            case 5:
                printf("Exiting...\n");
                break;

            default:
                printf("Invalid choice!\n");
        }
    } while(choice != 5);

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 3
Enter elements:
4
5
6

--- MENU ---
1. Insertion Sort
2. Merge Sort
3. Heap Sort
4. Quick Sort
5. Exit
Enter choice: 1
Sorted using Insertion Sort:
4 5 6
```

```
--- MENU ---
1. Insertion Sort
2. Merge Sort
3. Heap Sort
4. Quick Sort
5. Exit
Enter choice: 1
Sorted using Insertion Sort:
4 5 6

--- MENU ---
1. Insertion Sort
2. Merge Sort
3. Heap Sort
4. Quick Sort
5. Exit
Enter choice: 2
Sorted using Merge Sort:
4 5 6
```

```
--- MENU ---
1. Insertion Sort
2. Merge Sort
3. Heap Sort
4. Quick Sort
5. Exit
Enter choice: 3
Sorted using Heap Sort:
4 5 6

--- MENU ---
1. Insertion Sort
2. Merge Sort
3. Heap Sort
4. Quick Sort
5. Exit
Enter choice: 4
Sorted using Quick Sort:
4 5 6
```

```
--- MENU ---
1. Insertion Sort
2. Merge Sort
3. Heap Sort
4. Quick Sort
5. Exit
Enter choice: 5
Exiting...
```

Date:	
Program 10	Program to implement search algorithms - Linear & Binary search

```
#include <stdio.h>

// Linear Search
int linearSearch(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}

// Binary Search (array must be sorted)
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while(low <= high) {
        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int arr[100], n, key, choice, result;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\n--- MENU ---\n");
```

```
        printf("1. Linear Search\n");
        printf("2. Binary Search (Sorted Array Required)\n");
        printf("3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        printf("Enter element to search: ");
        scanf("%d", &key);

        switch(choice) {
            case 1:
                result = linearSearch(arr, n, key);
                if(result != -1)
                    printf("Element found at position %d\n", result + 1);
                else
                    printf("Element not found\n");
                break;

            case 2:
                result = binarySearch(arr, n, key);
                if(result != -1)
                    printf("Element found at position %d\n", result + 1);
                else
                    printf("Element not found\n");
                break;

            case 3:
                printf("Exiting...\n");
                break;

            default:
                printf("Invalid choice!\n");
        }

    } while(choice != 3);

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements:
2 4 6 8 10
```

```
--- MENU ---
1. Linear Search
2. Binary Search (Sorted Array Required)
3. Exit
Enter choice: 1
Enter element to search: 6
Element found at position 3

--- MENU ---
1. Linear Search
2. Binary Search (Sorted Array Required)
3. Exit
Enter choice: 2
Enter element to search: 8
Element found at position 4
```

```
--- MENU ---
1. Linear Search
2. Binary Search (Sorted Array Required)
3. Exit
Enter choice: 3
Exiting...
```

Date:	
Program 11	Program to implement various operations on Binary trees (Creation, Insertion, Deletion, Process)

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node *left, *right;
};

// Create node
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = newNode->right = NULL;
   return newNode;
}

// Insert (BST)
struct Node* insert(struct Node* root, int value) {
   if (root == NULL)
      return createNode(value);

   if (value < root->data)
      root->left = insert(root->left, value);
   else
      root->right = insert(root->right, value);

   return root;
}

// Find minimum (for deletion)
struct Node* findMin(struct Node* root) {
   while (root->left != NULL)
      root = root->left;
   return root;
}

// Delete node
struct Node* deleteNode(struct Node* root, int key) {
```

```
    if (root == NULL)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Case 1 & 2: One or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: Two children
        struct Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// Traversals (Process)
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node* root) {
```

```
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// MAIN
int main() {
    struct Node* root = NULL;
    int choice, value, n;

    do {
        printf("\n--- MENU ---\n");
        printf("1. Create Tree (Insert multiple)\n");
        printf("2. Insert\n");
        printf("3. Delete\n");
        printf("4. Inorder Traversal\n");
        printf("5. Preorder Traversal\n");
        printf("6. Postorder Traversal\n");
        printf("7. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter number of nodes: ");
                scanf("%d", &n);
                printf("Enter values:\n");
                for (int i = 0; i < n; i++) {
                    scanf("%d", &value);
                    root = insert(root, value);
                }
                break;

            case 2:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                root = insert(root, value);
                break;

            case 3:
                printf("Enter value to delete: ");
                scanf("%d", &value);
                root = deleteNode(root, value);
```

```
            break;

        case 4:
            printf("Inorder: ");
            inorder(root);
            printf("\n");
            break;

        case 5:
            printf("Preorder: ");
            preorder(root);
            printf("\n");
            break;

        case 6:
            printf("Postorder: ");
            postorder(root);
            printf("\n");
            break;

        case 7:
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid choice!\n");
    }

  } while (choice != 7);

  return 0;
}
```

OUTPUT:

```
--- MENU ---
1. Create Tree (Insert multiple)
2. Insert
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 1
Enter number of nodes: 5
Enter values:
5 3 7 2 4

Enter choice: 4
Inorder: 2 3 4 5 7

Enter choice: 2
Enter value to insert: 6
```

```
Enter choice: 2
Enter value to insert: 6

Enter choice: 4
Inorder: 2 3 4 5 6 7

Enter choice: 3
Enter value to delete: 3

Enter choice: 4
Inorder: 2 4 5 6 7

Enter choice: 7
Exiting...
```

Date:	
Program 12	Program to implement Binary tree traversals (Pre-order, In-order, Post-order)

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node *left, *right;
};

// Create node
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = newNode->right = NULL;
   return newNode;
}

// Insert (Binary Search Tree style)
struct Node* insert(struct Node* root, int value) {
   if(root == NULL)
      return createNode(value);

   if(value < root->data)
      root->left = insert(root->left, value);
   else
      root->right = insert(root->right, value);

   return root;
}

// Traversals
void preorder(struct Node* root) {
   if(root != NULL) {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void inorder(struct Node* root) {
```

```
    if(root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void postorder(struct Node* root) {
    if(root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// MAIN
int main() {
    struct Node* root = NULL;
    int choice, value, n;

    do {
        printf("\n--- MENU ---\n");
        printf("1. Insert Nodes\n");
        printf("2. Preorder Traversal\n");
        printf("3. Inorder Traversal\n");
        printf("4. Postorder Traversal\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter number of nodes: ");
                scanf("%d", &n);
                printf("Enter values:\n");
                for(int i = 0; i < n; i++) {
                    scanf("%d", &value);
                    root = insert(root, value);
                }
                break;

            case 2:
                printf("Preorder: ");
                preorder(root);
                printf("\n");
                break;
```

```
        case 3:
            printf("Inorder: ");
            inorder(root);
            printf("\n");
            break;

        case 4:
            printf("Postorder: ");
            postorder(root);
            printf("\n");
            break;

        case 5:
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid choice!\n");
    }

  } while(choice != 5);

  return 0;
}
```

OUTPUT:

```
--- MENU ---
1. Insert Nodes
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter choice: 1
Enter number of nodes: 5
Enter values:
1 2 3 4 5
```

```
--- MENU ---
Enter choice: 2
Preorder: 1 2 3 4 5

Enter choice: 3
Inorder: 1 2 3 4 5

Enter choice: 4
Postorder: 5 4 3 2 1

Enter choice: 5
Exiting...
```

Date:	
Program 13	Program to display the adjacency matrix of graph G.

```
//Program to obtain the adjacency matrix of a graph
#include <stdio.h>

#define MAX 100

int main()
{
   int adj[MAX][MAX];
   int vertices, edges;
   int src, dest;

   printf("Enter the number of vertices (max 100): ");
   scanf("%d", &vertices);

   printf("Enter the number of edges: ");
   scanf("%d", &edges);

   // Initialize matrix with zeros
   for (int i = 0; i < vertices; i++) {
      for (int j = 0; j < vertices; j++) {
         adj[i][j] = 0;
      }
   }

   // Input edges
   printf("\nEnter edges (source destination):\n");
   for (int i = 0; i < edges; i++) {
      printf("Edge %d: ", i + 1);
      scanf("%d %d", &src, &dest);

      // For directed graph
      adj[src][dest] = 1;

      // For undirected graph (uncomment next line)
      // adj[dest][src] = 1;
   }

   printf("Adjacency Matrix:\n");
   printf("    ");
   for (int i = 0; i < vertices; i++) {
```

```
        printf(" v%d ", i);
    }
    printf("\n");

    for (int i = 0; i < vertices; i++)
    {
        printf("v%d  ", i);
        for (int j = 0; j < vertices; j++)
        {
            printf(" %d  ", adj[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter the number of vertices (max 100): 5
Enter the number of edges: 4

Enter edges (source destination):
Edge 1: 0 1
Edge 2: 02
2
Edge 3: 0 3
Edge 4: 0 4
Adjacency Matrix:
     v0  v1  v2  v3  v4
v0   0   1   0   1   1
v1   0   0   0   0   0
v2   0   0   1   0   0
v3   0   0   0   0   0
v4   0   0   0   0   0
```

Date:	
Program 14	Program to implement traversal algorithms Breadth-First- Search/ Depth-First- Search traversal on a graph G

```
//Program to demonstrate Breadth First Search Traversal of a Graph.
#include <stdio.h>

#define MAX 100

int adj[MAX][MAX];
int visited[MAX];
int vertices;

// Queue implementation for BFS
int queue[MAX];
int front = -1, rear = -1;

void enqueue(int vertex) {
   if (rear == MAX - 1) {
      printf("Queue Overflow\n");
      return;
   }
   if (front == -1)
      front = 0;
   rear++;
   queue[rear] = vertex;
}

int dequeue() {
   if (front == -1 || front > rear) {
      printf("Queue Underflow\n");
      return -1;
   }
   int vertex = queue[front];
   front++;
   return vertex;
}

int isEmpty() {
   return (front == -1 || front > rear);
}

// Function to perform BFS
void BFS(int start) {
```

```
    // Reset visited array
    for (int i = 0; i < vertices; i++) {
        visited[i] = 0;
    }

    // Reset queue
    front = rear = -1;

    printf("%d ", start);
    visited[start] = 1;
    enqueue(start);

    while (!isEmpty()) {
        int current = dequeue();

        for (int i = 0; i < vertices; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                printf("%d ", i);
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}

// Function to display adjacency matrix in the required format
void displayMatrix() {

    printf("Adjacency Matrix:\n");
    printf("     ");
    for (int i = 0; i < vertices; i++) {
        printf(" v%d ", i);
    }
    printf("\n");

    for (int i = 0; i < vertices; i++) {
        printf("v%d   ", i);
        for (int j = 0; j < vertices; j++) {
            printf(" %d  ", adj[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int edges, src, dest, start;
```

```
    printf("Enter the number of vertices (max 100): ");
    scanf("%d", &vertices);

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    // Initialize matrix with zeros
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            adj[i][j] = 0;
        }
        visited[i] = 0;
    }

    // Input edges
    printf("\nEnter edges (source destination):\n");
    for (int i = 0; i < edges; i++) {
        printf("Edge %d: ", i + 1);
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;  // For directed graph
        // adj[dest][src] = 1;  // Uncomment for undirected graph
    }

    // Display adjacency matrix
    displayMatrix();

    // Get starting vertex for BFS
    printf("\nEnter starting vertex for BFS: ");
    scanf("%d", &start);

    // Perform BFS
    printf("\nBFS Traversal starting from vertex %d: ", start);
    BFS(start);
    printf("\n");

    return 0;
}
```

OUTPUT:

```
Enter the number of vertices (max 100): 4
Enter the number of edges: 4

Enter edges (source destination):
Edge 1: 0 1
Edge 2: 0 2
Edge 3: 1 2
Edge 4: 1 3
Adjacency Matrix:
      v0  v1  v2  v3
v0    0   1   1   0
v1    0   0   1   1
v2    0   0   0   0
v3    0   0   0   0

Enter starting vertex for BFS: 1

BFS Traversal starting from vertex 1: 1 2 3
```

Date:	
Program 15	Program to Implement B Tree for a given set of integers and perform operations insert(),search() and display() on the B Tree

```
#include <stdio.h>
#include <stdlib.h>

#define T 2   // Minimum degree

typedef struct BTree
{
   int n;                    // number of keys
   int leaf;                 // 1 if leaf, 0 otherwise
   int key[2 * T - 1];       // max keys = 3
   struct BTree *child[2 * T]; // max children = 4
} node;

node *root = NULL;

/* Create a new node */
node* createNode(int leaf)
{
   node *newNode = (node*)malloc(sizeof(node));
   newNode->leaf = leaf;
   newNode->n = 0;

   for (int i = 0; i < 2 * T; i++)
      newNode->child[i] = NULL;

   return newNode;
}

/* Traverse the B-Tree */
void traverse(node *ptr)
{
   if (ptr != NULL)
   {
      int i;
      for (i = 0; i < ptr->n; i++)
      {
         if (!ptr->leaf)
            traverse(ptr->child[i]);
         printf("%d ", ptr->key[i]);
      }
```

```
        if (!ptr->leaf)
            traverse(ptr->child[i]);
    }
}

/* Search a key in B-Tree */
node* search(node *ptr, int k)
{
    int i = 0;

    while (i < ptr->n && k > ptr->key[i])
        i++;

    if (i < ptr->n && k == ptr->key[i])
        return ptr;

    if (ptr->leaf)
        return NULL;

    return search(ptr->child[i], k);
}

/* Split the full child */
void splitChild(node *parent, int i, node *fullChild)
{
    node *newChild = createNode(fullChild->leaf);
    newChild->n = T - 1;

    for (int j = 0; j < T - 1; j++)
        newChild->key[j] = fullChild->key[j + T];

    if (!fullChild->leaf)
    {
        for (int j = 0; j < T; j++)
            newChild->child[j] = fullChild->child[j + T];
    }

    fullChild->n = T - 1;

    for (int j = parent->n; j >= i + 1; j--)
        parent->child[j + 1] = parent->child[j];

    parent->child[i + 1] = newChild;

    for (int j = parent->n - 1; j >= i; j--)
```

```
        parent->key[j + 1] = parent->key[j];

    parent->key[i] = fullChild->key[T - 1];
    parent->n = parent->n + 1;
}

/* Insert into non-full node */
void insertNonFull(node *ptr, int k)
{
    int i = ptr->n - 1;

    if (ptr->leaf)
    {
        while (i >= 0 && k < ptr->key[i])
        {
            ptr->key[i + 1] = ptr->key[i];
            i--;
        }

        ptr->key[i + 1] = k;
        ptr->n = ptr->n + 1;
    }
    else
    {
        while (i >= 0 && k < ptr->key[i])
            i--;

        i++;

        if (ptr->child[i]->n == 2 * T - 1)
        {
            splitChild(ptr, i, ptr->child[i]);

            if (k > ptr->key[i])
                i++;
        }

        insertNonFull(ptr->child[i], k);
    }
}

/* Insert a key */
void insert(int k)
{
    if (root == NULL)
    {
```

```
        root = createNode(1);
        root->key[0] = k;
        root->n = 1;
        return;
    }

    if (search(root, k) != NULL)
    {
        printf("Duplicate key not allowed.\n");
        return;
    }

    if (root->n == 2 * T - 1)
    {
        node *newRoot = createNode(0);
        newRoot->child[0] = root;

        splitChild(newRoot, 0, root);

        int i = 0;
        if (k > newRoot->key[0])
            i++;

        insertNonFull(newRoot->child[i], k);
        root = newRoot;
    }
    else
    {
        insertNonFull(root, k);
    }
}

/* Main */
int main()
{
    int ch, val;
    node *found;

    while (1)
    {
        printf("\n--- B-Tree Menu ---\n");
        printf("1. Insert\n");
        printf("2. Search\n");
        printf("3. Traverse\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
```

```
scanf("%d", &ch);

switch (ch)
{
   case 1:
      printf("Enter value to insert: ");
      scanf("%d", &val);
      insert(val);
      break;

   case 2:
      if (root == NULL)
      {
         printf("Tree is empty.\n");
      }
      else
      {
         printf("Enter value to search: ");
         scanf("%d", &val);
         found = search(root, val);

         if (found != NULL)
            printf("%d found in B-Tree.\n", val);
         else
            printf("%d not found in B-Tree.\n", val);
      }
      break;

   case 3:
      if (root == NULL)
         printf("Tree is empty.\n");
      else
      {
         printf("Traversal: ");
         traverse(root);
         printf("\n");
      }
      break;

   case 4:
      printf("Exiting...\n");
      exit(0);

   default:
      printf("Invalid choice.\n");
}
```

```
    }

    return 0;
}
```

OUTPUT:

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 5

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 7
```

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 5
Duplicate key not allowed.

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 6
```

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 2
Enter value to search: 5
5 found in B-Tree.

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 3
Traversal: 5 6 7
```

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 4
Exiting...
```

Date:	
Program 16	Program to store and retrieve student data from file using hashing. Use any collision resolution techniques

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 10

// Structure
struct Student {
   int id;
   char name[50];
};

// Hash table
struct Student table[SIZE];

// Initialize table
void init() {
   for(int i = 0; i < SIZE; i++) {
      table[i].id = -1;
      strcpy(table[i].name, "");
   }
}

// Hash function
int hash(int key) {
   return key % SIZE;
}

// Insert with Linear Probing
void insert(struct Student s) {
   int index = hash(s.id);

   int start = index;
   while(table[index].id != -1) {
      index = (index + 1) % SIZE;
      if(index == start) {
         printf("Hash table full!\n");
         return;
      }
   }
```

```
    table[index] = s;
}

// Save to file
void saveToFile() {
    FILE *fp = fopen("students.txt", "w");

    for(int i = 0; i < SIZE; i++) {
        if(table[i].id != -1) {
            fprintf(fp, "%d %s\n", table[i].id, table[i].name);
        }
    }

    fclose(fp);
    printf("Data saved to file.\n");
}

// Load from file
void loadFromFile() {
    FILE *fp = fopen("students.txt", "r");
    struct Student s;

    if(fp == NULL) {
        printf("File not found!\n");
        return;
    }

    init(); // clear table

    while(fscanf(fp, "%d %s", &s.id, s.name) != EOF) {
        insert(s);
    }

    fclose(fp);
    printf("Data loaded from file.\n");
}

// Search
void search(int id) {
    int index = hash(id);
    int start = index;

    while(table[index].id != -1) {
        if(table[index].id == id) {
            printf("Found: ID=%d Name=%s\n", table[index].id, table[index].name);
```

```
            return;
        }
        index = (index + 1) % SIZE;
        if(index == start) break;
    }

    printf("Record not found!\n");
}

// Display
void display() {
    printf("\nHash Table:\n");
    for(int i = 0; i < SIZE; i++) {
        if(table[i].id != -1)
            printf("[%d] ID=%d Name=%s\n", i, table[i].id, table[i].name);
        else
            printf("[%d] Empty\n", i);
    }
}

// MAIN
int main() {
    int choice, id;
    struct Student s;

    init();

    do {
        printf("\n--- MENU ---\n");
        printf("1. Insert Student\n");
        printf("2. Save to File\n");
        printf("3. Load from File\n");
        printf("4. Search Student\n");
        printf("5. Display Table\n");
        printf("6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {

            case 1:
                printf("Enter ID: ");
                scanf("%d", &s.id);
                printf("Enter Name: ");
                scanf("%s", s.name);
                insert(s);
```

```
            break;

        case 2:
            saveToFile();
            break;

        case 3:
            loadFromFile();
            break;

        case 4:
            printf("Enter ID to search: ");
            scanf("%d", &id);
            search(id);
            break;

        case 5:
            display();
            break;

        case 6:
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid choice!\n");
    }

  } while(choice != 6);

  return 0;
}
```

OUTPUT:

```
--- MENU ---
1. Insert Student
2. Save to File
3. Load from File
4. Search Student
5. Display Table
6. Exit
Enter choice: 1
Enter ID: 12
Enter Name: Ram

Enter choice: 1
Enter ID: 22
Enter Name: Shyam

Enter choice: 5
```

```
Hash Table:
[0] Empty
[1] Empty
[2] ID=12 Name=Ram
[3] ID=22 Name=Shyam
[4] Empty
...

Enter choice: 2
Data saved to file.

Enter choice: 3
Data loaded from file.

Enter choice: 4
Enter ID to search: 22
Found: ID=22 Name=Shyam
```